

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

ОТЧЕТ

по лабораторной работе N 1

**Создание серверного приложения на Node.js и
Express.js**

Тема проекта: платформа для обмена игровыми скриншотами и видео

Выполнил: студент группы _____

Проверил:

Минск 2026

1 Цель работы

Приобретение практических навыков создания серверного приложения на платформе Node.js с использованием фреймворка Express.js. Изучение маршрутизации, обработки HTTP-запросов GET, POST, PUT и DELETE, формирования JSON-ответов и обработки ошибок для последующей интеграции с клиентской частью проекта.

2 Краткое описание проекта

Разработан REST API для платформы обмена игровыми скриншотами и видео. Основной ресурс API - публикация (post). Публикация содержит идентификатор, заголовок, название игры, тип медиа, ссылку на файл, описание и массив текстовых комментариев. На данном этапе данные хранятся в массиве в оперативной памяти сервера.

Проект намеренно имеет простую структуру, соответствующую первой серверной лабораторной работе:

```
Lab1/
|-- server.js      # сервер Express и маршруты API
|-- requests.http  # готовые запросы для REST Client
|-- package.json   # зависимости и команды запуска
|-- package-lock.json
`-- output/pdf/report_lab1.pdf
```

3 Перечень маршрутов

Метод	Маршрут	Назначение	Статусы
GET	/posts	Получить все публикации; фильтр mediaType	200, 400
GET	/posts/:id	Получить публикацию по ID	200, 404
POST	/posts	Создать публикацию	201, 400
PUT	/posts/:id	Полностью обновить публикацию	200, 400, 404
DELETE	/posts/:id	Удалить публикацию	204, 404

Для отправки JSON используется заголовок Content-Type: application/json. Формат ошибки во всех маршрутах одинаков: { error: "сообщение" }.

4 Основные части файла server.js

Сервер подключает Express, включает разбор JSON, хранит публикации в массиве, проверяет входные данные и реализует CRUD-операции. Полный листинг файла приведен ниже.

```
1  const express = require('express');
2
3  const app = express();
4  const PORT = 3000;
5
6  app.use(express.json());
7
8  let posts = [
9    {
10     id: 1,
11     title: 'Sunset in Night City',
12     game: 'Cyberpunk 2077',
13     mediaType: 'screenshot',
14     mediaUrl: 'https://example.com/night-city.jpg',
15     description: 'A screenshot taken during an evening walk.',
16     comments: ['Great lighting!', 'This city looks amazing.'],
17   },
18   {
19     id: 2,
20     title: 'Winning goal',
21     game: 'Rocket League',
22     mediaType: 'video',
23     mediaUrl: 'https://example.com/winning-goal.mp4',
24     description: 'The final goal of the match.',
25     comments: ['Nice shot!'],
26   },
27 ];
28
29 let nextId = 3;
30
31 function validatePost(post) {
32   const requiredStringFields = ['title', 'game', 'mediaType', 'mediaUrl', 'description'];
33
34   for (const field of requiredStringFields) {
35     if (typeof post[field] !== 'string' || post[field].trim() === '') {
36       return `Field "${field}" must be a non-empty string`;
37     }
38   }
39
40   if (!['screenshot', 'video'].includes(post.mediaType)) {
41     return 'Field "mediaType" must be "screenshot" or "video"';
42   }
43
44   if (post.comments !== undefined) {
45     const commentsAreValid =
46       Array.isArray(post.comments) &&
47       post.comments.every((comment) => typeof comment === 'string');
48
49     if (!commentsAreValid) {
50       return 'Field "comments" must be an array of strings';
51     }
52   }
53
54   return null;
55 }
56
```

4 Основные части файла server.js (продолжение)

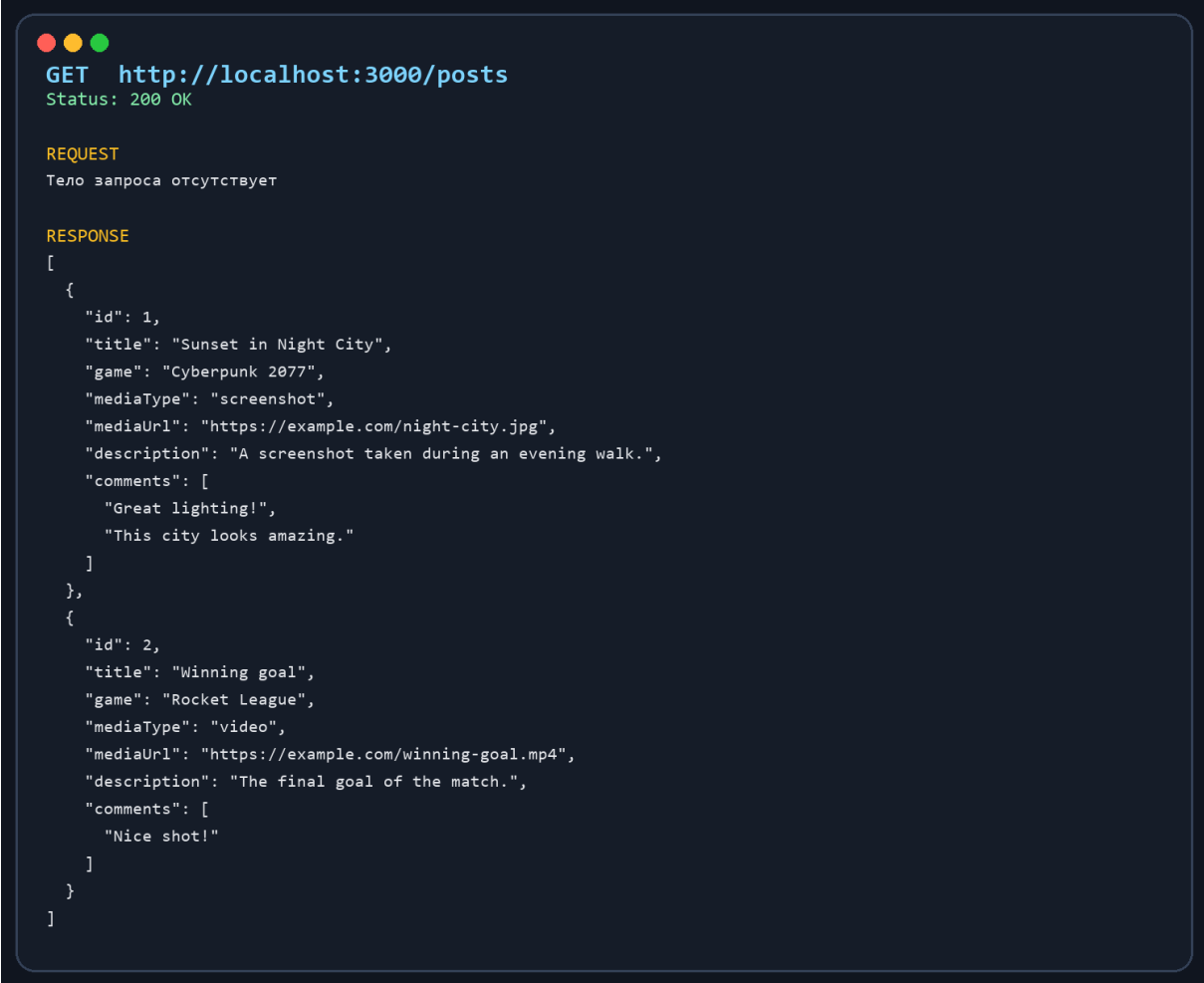
```
57 app.get('/', (req, res) => {
58   res.json({ message: 'Game Media API is running' });
59 });
60
61 app.get('/posts', (req, res) => {
62   const { mediaType } = req.query;
63
64   if (mediaType && ![ 'screenshot', 'video' ].includes(mediaType)) {
65     return res.status(400).json({
66       error: 'Query parameter "mediaType" must be "screenshot" or "video"',
67     });
68   }
69
70   const result = mediaType
71     ? posts.filter((post) => post.mediaType === mediaType)
72     : posts;
73
74   return res.json(result);
75 });
76
77 app.get('/posts/:id', (req, res) => {
78   const id = Number(req.params.id);
79   const post = posts.find((item) => item.id === id);
80
81   if (!post) {
82     return res.status(404).json({ error: 'Post not found' });
83   }
84
85   return res.json(post);
86 });
87
88 app.post('/posts', (req, res) => {
89   const validationError = validatePost(req.body);
90
91   if (validationError) {
92     return res.status(400).json({ error: validationError });
93   }
94
95   const newPost = {
96     id: nextId++,
97     title: req.body.title.trim(),
98     game: req.body.game.trim(),
99     mediaType: req.body.mediaType,
100    mediaUrl: req.body.mediaUrl.trim(),
101    description: req.body.description.trim(),
102    comments: req.body.comments || [],
103  };
104
105  posts.push(newPost);
106  return res.status(201).json(newPost);
107 });
108
109 app.put('/posts/:id', (req, res) => {
110   const id = Number(req.params.id);
111   const postIndex = posts.findIndex((item) => item.id === id);
112
```

4 Основные части файла server.js (продолжение)

```
113   if (postIndex === -1) {
114     return res.status(404).json({ error: 'Post not found' });
115   }
116
117   const validationError = validatePost(req.body);
118
119   if (validationError) {
120     return res.status(400).json({ error: validationError });
121   }
122
123   const updatedPost = {
124     id,
125     title: req.body.title.trim(),
126     game: req.body.game.trim(),
127     mediaType: req.body.mediaType,
128     mediaUrl: req.body.mediaUrl.trim(),
129     description: req.body.description.trim(),
130     comments: req.body.comments || [],
131   };
132
133   posts[postIndex] = updatedPost;
134   return res.json(updatedPost);
135 });
136
137 app.delete('/posts/:id', (req, res) => {
138   const id = Number(req.params.id);
139   const postIndex = posts.findIndex((item) => item.id === id);
140
141   if (postIndex === -1) {
142     return res.status(404).json({ error: 'Post not found' });
143   }
144
145   posts.splice(postIndex, 1);
146   return res.status(204).send();
147 });
148
149 app.use((req, res) => {
150   res.status(404).json({ error: 'Route not found' });
151 });
152
153 app.use((err, req, res, next) => {
154   console.error(err);
155
156   if (err instanceof SyntaxError && err.status === 400) {
157     return res.status(400).json({ error: 'Invalid JSON' });
158   }
159
160   res.status(500).json({ error: 'Internal server error' });
161 });
162
163 app.listen(PORT, () => {
164   console.log(`Server is running at http://localhost:${PORT}`);
165 });
```

5 Тестирование API

Сервер запущен командой `npm run dev`. Запросы выполнены последовательно по адресу `http://localhost:3000`. Ниже приведены зафиксированные запросы, HTTP-статусы и ответы. После DELETE выполнен повторный GET, а также отправлен неполный объект для проверки статуса 400.



```
GET http://localhost:3000/posts
Status: 200 OK

REQUEST
Тело запроса отсутствует

RESPONSE
[
  {
    "id": 1,
    "title": "Sunset in Night City",
    "game": "Cyberpunk 2077",
    "mediaType": "screenshot",
    "mediaUrl": "https://example.com/night-city.jpg",
    "description": "A screenshot taken during an evening walk.",
    "comments": [
      "Great lighting!",
      "This city looks amazing."
    ]
  },
  {
    "id": 2,
    "title": "Winning goal",
    "game": "Rocket League",
    "mediaType": "video",
    "mediaUrl": "https://example.com/winning-goal.mp4",
    "description": "The final goal of the match.",
    "comments": [
      "Nice shot!"
    ]
  }
]
```

Рисунок 1 - Получение списка публикаций



Рисунок 2 - Получение публикации по идентификатору



```

POST http://localhost:3000/posts
Status: 201 Created

REQUEST
{
  "title": "Boss fight",
  "game": "Elden Ring",
  "mediaType": "video",
  "mediaUrl": "https://example.com/boss-fight.mp4",
  "description": "A short video from the boss fight.",
  "comments": [
    "Well played!"
  ]
}

RESPONSE
{
  "id": 3,
  "title": "Boss fight",
  "game": "Elden Ring",
  "mediaType": "video",
  "mediaUrl": "https://example.com/boss-fight.mp4",
  "description": "A short video from the boss fight.",
  "comments": [
    "Well played!"
  ]
}

```

Рисунок 3 - Создание публикации



```
● ● ●
PUT http://localhost:3000/posts/3
Status: 200 OK

REQUEST
{
  "title": "Boss fight - no damage",
  "game": "Elden Ring",
  "mediaType": "video",
  "mediaUrl": "https://example.com/boss-fight-no-damage.mp4",
  "description": "The boss was defeated without taking damage.",
  "comments": [
    "Impressive!",
    "Great timing!"
  ]
}

RESPONSE
{
  "id": 3,
  "title": "Boss fight - no damage",
  "game": "Elden Ring",
  "mediaType": "video",
  "mediaUrl": "https://example.com/boss-fight-no-damage.mp4",
  "description": "The boss was defeated without taking damage.",
  "comments": [
    "Impressive!",
    "Great timing!"
  ]
}
```

Рисунок 4 - Полное обновление публикации



Рисунок 5 - Удаление публикации

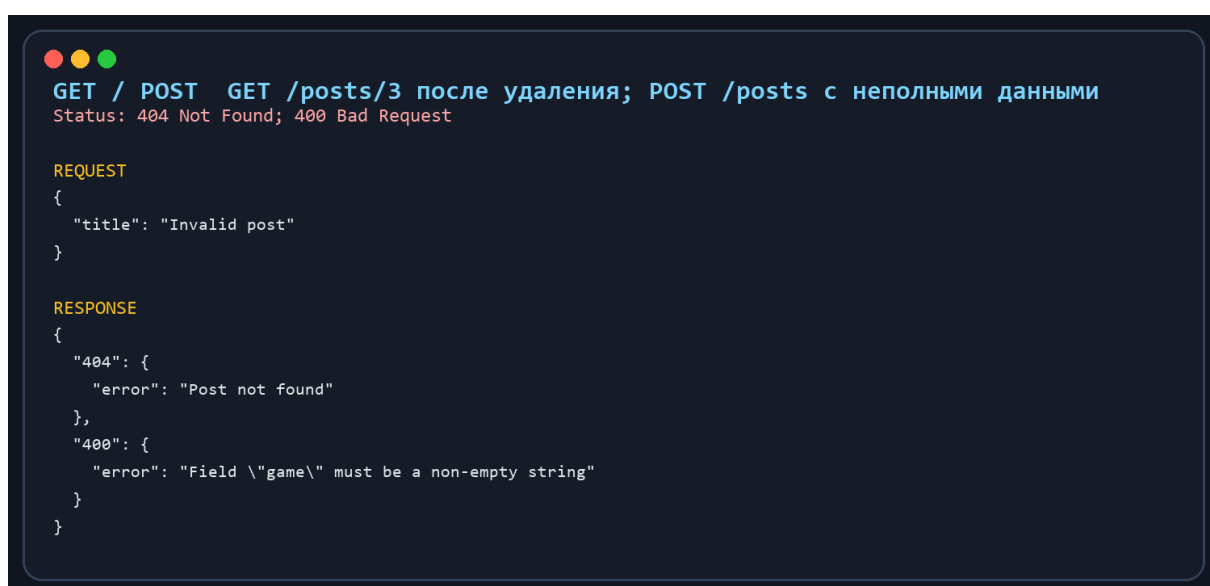


Рисунок 6 - Проверка ответов с ошибками

6 Ответы на контрольные вопросы

1. Какие методы HTTP реализованы и для каких операций?

GET используется для чтения списка и одной публикации, POST - для создания, PUT - для полного обновления, DELETE - для удаления.

2. Как получить route params и query string в Express?

Параметры маршрута доступны в объекте req.params, например req.params.id. Параметры строки запроса доступны в req.query, например req.query.mediaType.

3. Для чего используется middleware express.json()?

Он разбирает тело запроса в формате JSON и помещает полученный JavaScript-объект в req.body.

4. Какие статусы возвращаются при создании и удалении?

При успешном создании возвращается 201 Created. При успешном удалении без тела ответа - 204 No Content.

5. Что такое REST API и каковы его основные принципы?

REST API - интерфейс взаимодействия с ресурсами через HTTP. Основные принципы: клиент-серверное разделение, отсутствие состояния сеанса на сервере между запросами, единообразный интерфейс, возможность кэширования, слоистая архитектура и представление ресурсов через URI.

6. Как организовать глобальную обработку ошибок в Express?

В конце цепочки middleware добавляют функцию с четырьмя параметрами: (err, req, res, next). Она перехватывает переданные ошибки и формирует единый HTTP-ответ.

7. Какая структура ответа выбрана для ошибок?

Используется простой JSON-объект вида { error: "message" }. Такая структура одинакова для 400, 404 и 500.

8. Как протестировать DELETE, если ресурс уже удален?

Повторно отправить DELETE или GET с тем же ID. Сервер не должен находить ресурс и должен вернуть 404 Not Found с JSON-описанием ошибки.

9. Какие преимущества дает nodemon?

Nodemon следит за изменениями файлов и автоматически перезапускает сервер. Разработчику не нужно останавливать и запускать Node.js вручную после каждой правки.

10. В чем разница между PUT и PATCH?

PUT заменяет полное представление ресурса, поэтому клиент отправляет все поля. PATCH изменяет только указанные поля. В работе реализован PUT, так как он прямо указан в задании и демонстрирует полное обновление.

11. Как добавить поддержку CORS?

Установить пакет cors командой npm install cors, подключить его через require('cors') и добавить app.use(cors()). При необходимости разрешенные источники и методы задаются в настройках middleware.

12. Почему данные находятся в массиве, а не в базе данных?

Массив позволяет сосредоточиться на Express, маршрутах и HTTP без лишней настройки. База данных нужна, когда требуется постоянное хранение, большой объем данных, параллельная работа пользователей, поиск, связи и надежность.

13. Какой Content-Type нужен для JSON?

Клиент должен отправить заголовок Content-Type: application/json. Express также автоматически формирует этот тип для ответа res.json().

14. Пример curl-команды для POST.

```
curl -X POST http://localhost:3000/posts -H "Content-Type: application/json" -d '{"title":"Boss fight","game":"Elden Ring","mediaType":"video","mediaUrl":"https://example.com/video.mp4","description":"Boss battle","comments":[]}'
```

15. Как проверять сервер после каждой модификации?

Запускать сервер через `npm run dev`, наблюдать отсутствие ошибок запуска, повторять запросы из `requests.http` и проверять статус и JSON-ответ. Для повторяемой проверки в будущем можно добавить автоматические тесты.

16. Как запустить приложение на другом компьютере?

Установить Node.js, скопировать или клонировать репозиторий, перейти в папку Lab1, выполнить `npm install` и `npm start`. После запуска API будет доступен на порту 3000. Папку `node_modules` переносить не требуется.

7 Вывод

В ходе лабораторной работы создано серверное приложение на Node.js с использованием Express.js для предметной области обмена игровыми скриншотами и видео. Реализованы маршруты GET, POST, PUT и DELETE для ресурса posts, разбор JSON, проверка входных данных, ответы со статусами 200, 201, 204, 400, 404 и 500, а также глобальный обработчик ошибок. Все основные маршруты проверены последовательными HTTP-запросами. Полученная основа может быть расширена подключением базы данных, аутентификацией и интеграцией с клиентской частью в следующих лабораторных работах.